Performance Impact of Microservice Granularity Decisions: An Empirical Evaluation Using the Service Weaver Framework

Ricardo César Mendonça Filho[1] and Nabor C. Mendonça[2][0000−0002−2972−9423]

[1] Center of Technological Sciences
[2] Post Graduate Program in Applied Informatics
University of Fortaleza
Fortaleza, Ceará, Brazil
{ricardocesarfilho,nabor}@unifor.br

Abstract. The Service Weaver framework enables the development of distributed applications in Go as modular monoliths, with the flexibility to deploy monolith components in different environments and at various levels of granularity without code changes. This work evaluates the performance of an open-source microservice application using Service Weaver, considering multiple service granularity decisions in environments of one and two virtual machines under various workloads. The results indicate that service decoupling, while beneficial to modularity and maintenance, can significantly increase communication overhead between processes and virtual machines, negatively affecting the application's performance and scalability. These findings highlight the importance of balancing service granularity and communication costs in the design and deployment of microservice applications.

Keywords: Microservice granularity · Performance analysis.

1 Introduction

One of the crucial challenges in designing microservices-based applications is determining the ideal level of granularity for each service [10, 27]. If the services are too large, the benefits of loose coupling and independence are lost. If they are too small, there may be unnecessary complexity and communication overhead, affecting the performance and scalability of the application [15]. Traditional solutions used in microservices development are not suitable to face these challenges [5]. The main reason is that they all expect the developer to manually decompose the application into several independent services at design time. This means that the communication topology of the application is predetermined by the application developer. In practice, this constraint reduces the granularity options that can be explored by microservices developers, forcing them to make sub-optimal design decisions prematurely [4].

The Service Weaver framework, launched by Google in early 2023, is an open-source solution with the potential to address the granularity challenges

discussed above [6]. With Service Weaver, developers can write their distributed applications in Go as a modular monolith [25]. Later, at deployment time, the framework splits this monolith into multiple distributed services, according to the communication topology defined by the developer, without the need for changes in the monolith's code. This allows the application's services to be easily deployed locally or in the cloud, under different communication topologies, with minimal configuration effort [5]. In this sense, Service Weaver innovates by combining the advantages of a modular monolith with the flexibility and scalability of microservices [11].

Our work leverages the Service Weaver framework to explore key issues in microservice architecture design and deployment concerning possible service granularity decisions. To this end, we aim to answer the following research questions:

RQ1 What is the impact on the performance of a microservice application when the application's services are deployed with different levels of granularity using Service Weaver?

RQ2 How is the observed granularity impact influenced by factors such as the expected workload and the allocation of services to the resources of the underlying infrastructure?

To answer these two questions, we: (i) selected a widely adopted open-source microservice demo application as a research *benchmark*; and (ii) utilized the flexible service deployment features originally introduced by the Service Weaver framework to test the performance of the target microservice application considering different service deployment configurations, under different workloads, in scenarios with one and two virtual machines (VMs).

Our experimental results reveal that the choice of service granularity in microservices architectures has a significant impact on performance, especially regarding communication strategies between processes located on the same VM or different VMs. The deployment configurations tested on a single VM indicated that, although service decoupling can offer initial benefits in terms of modularity and maintenance, the overhead of inter-process communication can become a limiting factor under high workloads. In contrast, configurations distributed across two VMs showed that VM-to-VM communication introduces an additional layer of complexity and latency, particularly under high load conditions. These findings demonstrate that architectural decisions regarding service granularity and the allocation of services to the underlying computing infrastructure are critical aspects to be considered in the design and deployment of microservices applications, directly influencing their performance and scalability.

The rest of this work is organized as follows: Section 2 provides an overview of the Service Weaver framework; Section 3 describes the experimental evaluation method used; Section 4 presents and analyzes the obtained results; Section 5 discusses the limitations and broader implications of our findings; Section 6 compares our work with other related studies; finally, Section 7 offers some concluding remarks and suggests topics for future research.

2 Service Weaver

Service Weaver is a framework developed to facilitate the development, deployment, and management of distributed applications in Go [6]. The two main innovations introduced by the framework are (i) a programming model that allows developers to write modular distributed applications focused solely on business rules, compiled into a single binary artifact, and (ii) a runtime environment for the optimized distribution and management of these applications [5].

The proposed programming model allows the programmer to write a distributed application as a single program, where the code is divided by modular units, called *components*. The framework decouples the implementation of business rules from the configuration and management of infrastructure resources: the components are centered around logical boundaries based on the application's business rules, and the framework's runtime environment is centered around physical boundaries based on application performance (for example, two components may be co-located to improve performance). This decoupling, along with the fact that physical boundaries can be dynamically changed, helps to alleviate the challenges of modularity and granularity caused by architecturally rigid APIs, common in modern distributed architectures [5].

At deployment time, the Service Weaver's runtime environment analyzes the application's source code, generating the necessary code to handle remote calls between services, allowing them to be executed on different processes or machines without the need for explicit implementation of remote calls by the developer. Additionally, the framework manages the scheduling, replication, and co-location of the created services, without requiring developer intervention [6].

2.1 Components

A Service Weaver application is composed of a set of components. A component is described as an interface implemented in Go, and components can communicate with each other through method calls defined in these interfaces. This feature allows for the abstraction of remote calling mechanisms and data serialization, which do not need to be explicitly implemented by the application developer [6].

Components can be hosted on different operating system processes and possibly on multiple machines. Method invocations of components turn into remote procedure calls when necessary, but remain as local procedure calls if the calling and called components are in the same process [5].

2.2 Deployment Modes

Service Weaver does not include specific code for a particular deployment environment. Instead, the framework uses a special API, called *Deployer*, to integrate the application logic with the details of the execution environment, such as the physical distribution in multiple virtual machines in a public cloud. This deployment API allows developers to write Go applications that are independent of

the deployment environment. The latest version of Service Weaver offers various types of Deployers that allow deploying application components in different execution environments, including local, Kubernetes and Google Cloud Deployers [6].

2.3 Deployment Configuration

Service Weaver provides a high-level notation for developers to explicitly define aspects of the application's infrastructure and communication to be considered at deployment time, such as which components should run locally, in a single process, or be distributed in multiple processes, possibly on multiple machines. Moreover, developers can define more complex deployment configurations, such as deploying components in different virtual machines, by customizing the configuration files Service Weaver automatically generates for each specific deployment environment. For example, a developer can customize the configuration files for a Kubernetes deployment by specifying which Deployment should run in which Kubernetes node.[3] These features give application developers more flexibility to experiment with different levels of service granularity as well as to assess the performance impact of each granularity decision under multiple deployment environments. Further details on the deployment configuration features of Service Weaver can be found in the official documentation [6].

3 Evaluation Method

This section presents the evaluation method adopted to answer the two research questions introduced in Section 1, including the target microservices application, the testing environment, and the performance metric and load generation tool used.

3.1 Target Application

The target application chosen was Online Boutique [7]. This application implements a virtual store where users can search for products, add products to the cart, and purchase them. It was originally developed by Google as a way to demonstrate a set of cloud-native technologies, such as Kubernetes, GKE, Istio, and gRPC. Since then, it has been used as a microservice evaluation benchmark in several research studies reported in the literature (e.g., [3, 23, 17, 20]).

The architecture of Online Boutique consists of 11 microservices, responsible for different aspects of the virtual store's operation, as illustrated in Figure 1. In the original version of the application, these services are implemented in different programming languages, such as Go, Java, C#, Python, and Node.js.

[3] When using the Kubernetes deployer, Service Weaver creates a separate Deployment specification for each group of co-located components specified in the application's configuration file [6].

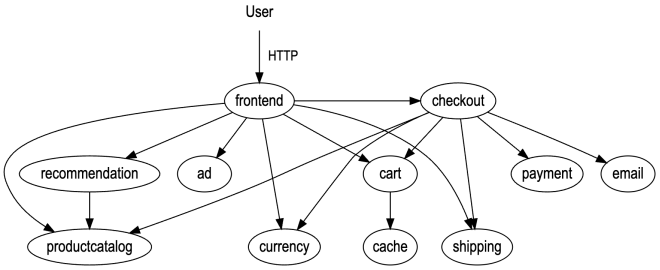


Fig. 1. Architecture of the Online Boutique application (source: [7]).

To enable the application to be executed using Service Weaver, the framework's developers ported the remaining (non-Go) Online Boutique services to the Go language, in addition to replacing the use of the Redis cache service with a native Go version [5]. This new version of the application, written entirely in Go, was used in the experiments conducted in this work.

From the architectural diagram shown in Figure 1, it is evident that the *frontend* service serves as the entry point of the application. This service plays the dual role of implementing the virtual store's web pages, as well as orchestrating the other services.

3.2 Testing Environment

All tests were conducted on a single host machine with the following characteristics: AMD Ryzen 7 5800H 3.2GHz processor, 8 processing cores, and 16GB of DDR4 RAM. To better represent potential real-world deployment scenarios, we deployed the application in the host machine in configurations involving both a single VM and a dual VM setup. To this end, we created a Kubernetes cluster using Minikube [26] containing two virtual machines created with VirtualBox [16]. Each VM was standardized with 4 CPUs and 4GB of RAM to maintain consistency across tests, allowing us to isolate the impact of service distribution from hardware variability.

The deployment process of the target application in this cluster was carried out using the configuration files automatically generated by the Kubernetes deployer provided by Service Weaver. These files are dynamically modified to adapt to the desired architectures, allowing for the creation of specific infrastructure conditions for each test scenario. All performance measurements were obtained from the actual execution of the application under different configurations on the local Kubernetes cluster. No artificial communication delays were introduced to differentiate local calls from remote calls between services.

The decision to use a single host machine with only two virtual machines as the testing infrastructure was motivated by three factors. First, the target application is relatively small, with only 11 non-persistent services, making it possible to deploy all services in a single process. Second, the use of a single dedicated

host machine allows for more controlled performance testing, enabling a more accurate analysis of the performance impact resulting from the deployment granularity of the application's services. Finally, the use of only two virtual machines simplifies the configuration and execution of tests, reducing the complexity and time required for the experiments.

3.3 Load Generation and Performance Metric

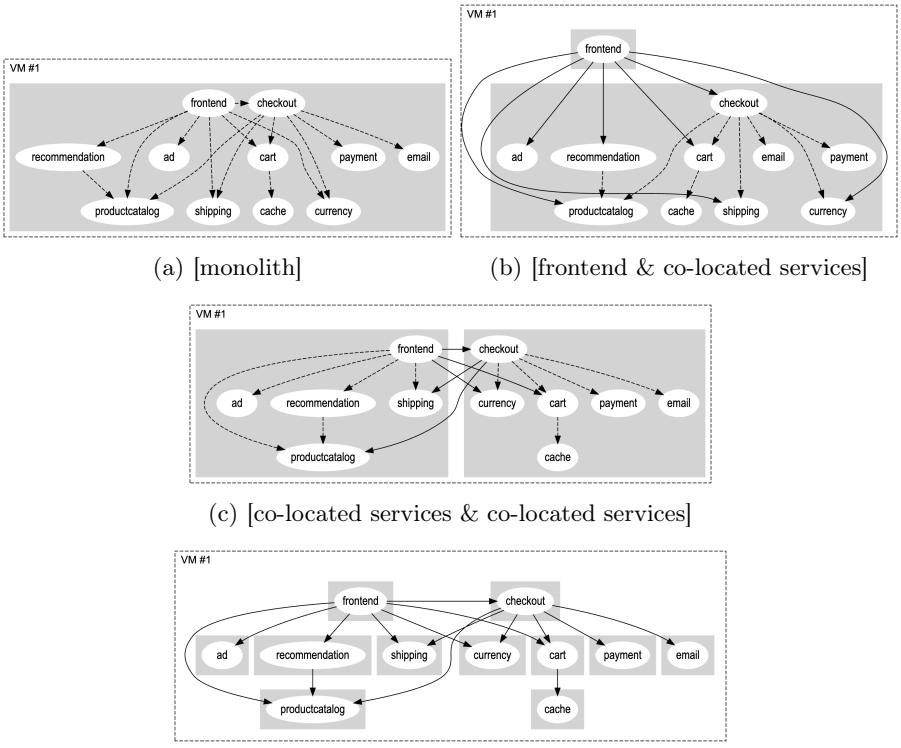
The Locust [13] load generation tool was used to simulate different levels of workload on the target application. In addition to being widely used in web application load testing, the choice of Locust was a natural decision, as the original Online Boutique application comes with a Locust load testing script. This script invokes a series of operations offered by the application's *frontend* service, simulating the behavior of a typical store user:

- opening the store's homepage;
- performing two currency exchanges;
- conducting ten product searches;
- adding two products to the shopping cart;
- visualizing the cart items three times; and
- completing the purchases (checkout).

Each load test involved running Locust with the above script for about 5 minutes, with a growing number of virtual users, starting with 500 users up to the limit of 2250 users, in increments of 250 users. These load values were defined after conducting a set of initial tests, where we determined the maximum number of virtual users supported by the application when deployed on the previously described physical infrastructure.

At the end of each test, Locust produces a set of performance metrics. These metrics include a variety of statistics calculated on the response times measured for each operation requested from the target application, such as minimum, maximum, average, median, and various percentiles, as well as the throughput and error rate of each operation. Locust also calculates consolidated statistics, considering the set of operations performed during the test in an aggregated manner.

For the experiments conducted in this work, the performance metric used was the 95th percentile of response times, in milliseconds, calculated in an aggregated manner over the set of operations requested from the application by each user. The choice of a high percentile (>90%) as the performance metric is strategically advantageous due to its ability to realistically capture the application's behavior under stress conditions [14]. Unlike more traditional metrics such as average and median, high percentile metrics avoid distortions caused by outliers, which tend to occur more frequently in overload scenarios.



(a) [monolith]

(b) [frontend & co-located services]

(c) [co-located services & co-located services]

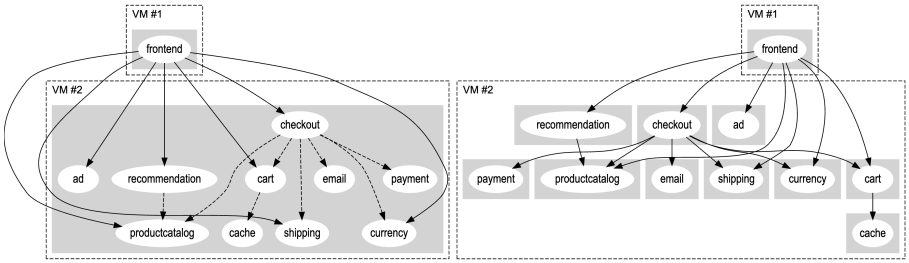
(d) [distributed services]

Fig. 2. Deployment configurations of the Online Boutique application in one VM. Dashed rectangles represent virtual machines, filled grey rectangles represent processes, white ellipses represent application components, dashed arrows represent in-process local calls, and solid arrows represent remote calls.

3.4 Selected Deployment Configurations

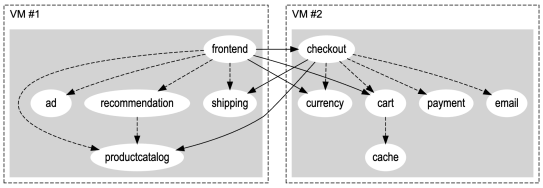
To investigate how service granularity can affect the performance of a microservice application, we used the deployment features of Service Weaver to define eight deployment configurations out of the eleven Online Boutique microservices, considering scenarios with one and two virtual machines.

Figures 2 and 3 illustrate these eight configurations, with four deployed on a single virtual machine (Figure 2) and four on two virtual machines (Figure 3). In all configuration diagrams shown in Figures 2 and 3, unfilled rectangles with dashed borders represent virtual machines, filled grey rectangles represent processes, and white ellipses represent application components. Dashed arrows represent local calls between components executed within the same process, while solid arrows represent remote calls between components executed in different processes.

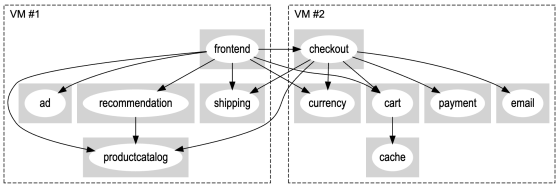


(a) [frontend] & [co-located services]

(b) [frontend] & [distributed services]



(c) [co-located services] & [co-located services]



(d) [distributed services] & [distributed services]

Fig. 3. Deployment configurations of the Online Boutique application in two VMs.

The chosen configurations range from a monolithic configuration (Figure 2(a)), executed on a single virtual machine, to fully distributed configurations (Figures 3(b) and 3(d)), executed on two virtual machines. Moreover, some configurations were designed to isolate a specific service of the application from the other services, such as the configurations that isolate the *frontend* service, illustrated in Figures 2(b), 3(a), and 3(b). These three configurations were selected given the importance of the *frontend* service as the application's entry point. The configuration names were defined in a way to represent the strategy used to deploy the eleven application services, with brackets used to distinguish the services deployed in each VM. For example, the name [frontend & co-located services] (Figure 2(b)) indicates that the *frontend* service is deployed isolated in a single process, while the remaining services are co-located in a second process, with both processes on a single VM. Similarly, the name [distributed services] & [distributed services] (Figure 3(d)) indicates that all services are fully distributed across the two VMs, with each VM containing a subset of the application's services.

3.5 Considerations on the Number of Possible Configurations

The eight deployment configurations selected in our study comprise only a small subset of all possible configurations that could be used to deploy the eleven microservices of the target application to the two VMs. The problem of calculating the total number of possible deployment configurations of a given number of services in a single VM can be solved using combinatorics, as the number of ways to partition a set of n elements into non-empty subsets. This number is given by the Bell numbers, named after Eric Temple Bell [19]. A Bell number, denoted as B_n, represents the number of different ways to partition a set of n elements. The sequence of Bell numbers begins $B_0, B_1, B_2, \ldots$ with values $1, 1, 2, 5, 15, \ldots$. For example, the set $\{1, 2, 3\}$ can be partitioned in non-empty subsets in five different ways ($B_3 = 5$): {{1}, {2}, {3}}, {{1, 2}, {3}}, {{1, 3}, {2}}, {{2, 3}, {1}}, and {{1, 2, 3}}.

Bell numbers can be calculated, among other methods, using the recursive formula [19]:

$$B_{n+1} = \sum_{k=0}^{n} \binom{n}{k} B_k$$

where $B_0 = 1$.

For the case of the Online Boutique application, with eleven services, the number of possible deployment configurations in a single VM is given by the Bell number B_{11}, which is equal to 678,570. This number represents the total number of ways to partition the eleven services into non-empty subsets, considering all possible combinations of service granularity decisions. The number of possible deployment configurations in multiple VMs is much greater, as it involves partitioning the eleven services into two non-empty subsets, with each subset deployed in either one of the VMs. For example, if a partition consists of k subsets, the number of ways to allocate these k subsets across two VMs is $2^{k-1} - 1$, reflecting the binary decision for each subset to join one of the two VMs, excluding the case where all subsets end up in one VM. This means that there are $2^{11-1} - 1 = 1023$ possible ways to deploy the eleven Online Boutique microservices across two VMs in a fully distributed fashion, i.e., with each service in a separate process, which is only one out of the 678,570 possible ways the application's microservices can be deployed in terms of granularity choices.

The magnitude of these numbers highlights the complexity of choosing an optimal deployment strategy even for a small number of services and virtual machines. Finding the optimal deployment configuration for a given microservice application across a cluster of VMs is a challenging problem, as it involves balancing the trade-offs between service granularity, communication overhead, and resource allocation, among other factors, in an exponentially large deployment space. The experimental evaluation conducted in this work aims to shed an early light on these trade-offs by analyzing the performance of the Online Boutique application under eight representative deployment configurations, considering scenarios with only one or two VMs.

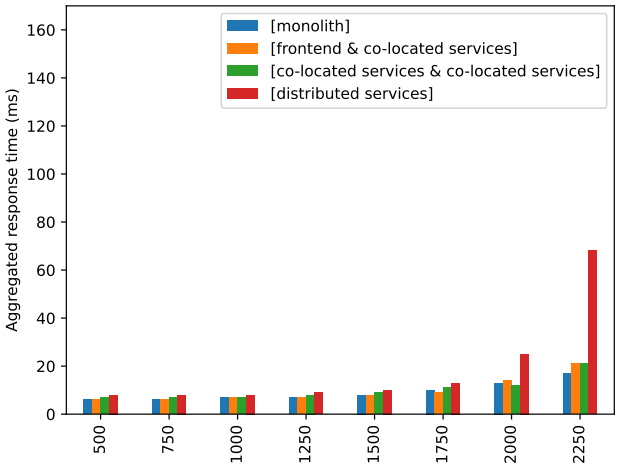


Fig. 4. Performance of the four deployment configurations in one VM.

4 Results

Figures 4 and 5 show the performance of the eight deployment configurations of the target application for the scenarios with one and two VMs, respectively. Each figure displays a plot for the 95th percentile of the application's aggregated response times measured in milliseconds for each configuration, under different workloads.

4.1 Configurations in One VM

Figure 4 displays the performance of the four deployment configurations on a single VM. The monolithic configuration showed a progressive increase in response time, from 6 ms for 500 users to 17 ms for 2250 users, indicating scalability limitations under larger loads due to its centralized nature. The [frontend & co-located services] configuration, with response times increasing from 6 ms to 21 ms, had a similar initial performance but suffered a slightly more pronounced increase under the higher load, likely caused by the greater number of remote invocations resulting from the decoupling of the *frontend* service from the other services. The [co-located services & co-located services] configuration showed a similar trend to the previous ones, with an initial response time of 7 ms and an increase to 21 ms, indicating that distributing the services into two coupled groups offers some benefit in terms of load distribution, but still faces similar scalability challenges due to remote invocations between the two groups of services. Finally, the [distributed services] configuration, with response times ranging from 8 ms to 68 ms, showed performance similar to the other configurations initially but experienced a significant increase at the highest load, the largest among the four single-VM

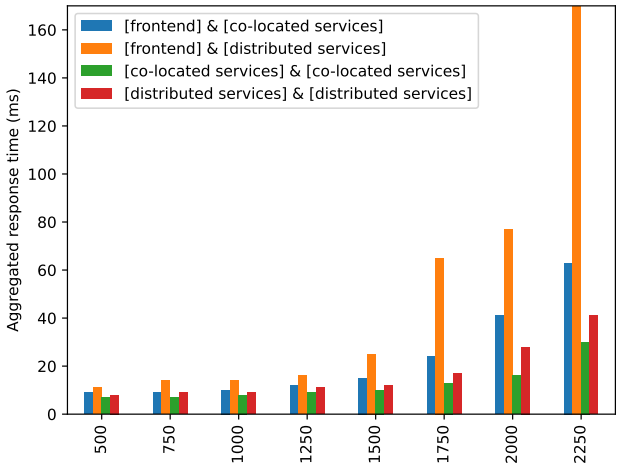


Fig. 5. Performance of the four deployment configurations in two VMs.

configurations. The fully distributed nature of the services in this configuration, although offering greater flexibility and initial scalability potential, ultimately deteriorates its performance due to excessive communication overhead among the remote services.

4.2 Configurations in Two VMs

The configurations in two VMs (Figure 5) showed greater variability in results, reflecting the greater complexity of distributing services in this deployment scenario. The [frontend] & [co-located services] configuration displayed an increase in response time from 9 ms to 63 ms, indicating that isolating the *frontend* on a separate VM became a bottleneck at high loads. The [frontend] & [distributed services] configuration started with a response time of 11 ms, increasing dramatically to 170 ms under maximum load, by far the worst performance among all the evaluated configurations. This result suggests that the communication overhead among fully distributed services on separate VMs is a critical factor at high loads. The [co-located services] & [co-located services] configuration offered the best scalability among the two VM configurations, with an increase from 7 ms to 30 ms, similar to the scalability of the best one VM configurations. Distributing the services into two coupled groups on separate VMs appears to offer a good balance between service granularity and communication overhead between them, resulting in a gradual and controlled increase in response time. Finally, the [distributed services] & [distributed services] configuration performed similarly to the [frontend] & [co-located services] configuration, with response times increasing from 8 ms to 41 ms, revealing that the initial flexibility offered by total decoupling between services on the two VMs faces significant communication

challenges between VMs under high loads, although with a lesser impact than the distributed configurations with the *frontend* service isolated on the first VM.

4.3 Joint Analysis of the Two Deployment Scenarios

Comparing the performance of all eight configurations, it is possible to identify some general trends about the impact of service granularity and communication overhead between machines. In the one VM configurations, there is a progressive increase in response times with increasing load, particularly evident in the fully distributed configurations, where the communication overhead among processes becomes a limiting factor. In contrast, the two VM configurations demonstrate that the physical distribution of services across different machines can introduce an additional cost of communication between services. Particularly, configurations with services fully distributed on two VMs suffer a more pronounced increase in response times under high loads, indicating that communication among processes located on the same VM may be more critical than communication among processes located on different VMs. On the other hand, configurations with co-located services on two VMs performed similarly to configurations with co-located services on a single VM, indicating that communication between VMs may be less critical when services are co-located within each VM.

These results suggest that while service granularity offers scalability benefits at lower loads, the complexity and overhead associated with remote invocations, especially in configurations with services distributed across multiple VMs, become dominant factors under high loads. Thus, the choice between different microservice granularity decisions should consider a balance between the flexibility offered by the physical distribution of services and the performance implications associated with communication strategies among processes of the same VM or different VMs.

5 Discussion

Our work is an early step toward understanding the performance impact of service granularity when designing and deploying microservice applications. As such, it has several limitations that should be considered when interpreting our results. First, the results were derived from experiments conducted on a single demo application, the Online Boutique, with a single performance metric, aggregated response time. While this application and metric were selected to be representative of a real-world microservices architecture, different applications measured using different performance metrics could yield different results. In that direction, a more fine-grained analysis of the response time and resource consumption of individual services could provide further insights into the performance bottlenecks and their root causes. Second, the experiments were conducted in controlled environments with a limited number of virtual machines, which may not fully capture the performance dynamics of more complex cloud

environments with elastic scaling capabilities. Finally, the experiments were limited to a predefined set of communication patterns established by the Service Weaver framework. Future research could explore the impact of deploying the application in more realistic cloud environments using alternative communication strategies, such as event-driven approaches, which might affect the inter-service communication dynamics.

Despite these limitations, our work contributes to a deeper understanding of several broader implications for microservices architecture design. In particular, our results suggest that the choice of service granularity can significantly impact the performance and scalability of a microservice application, with trade-offs between the benefits of decoupling and the performance penalties due to increased communication overhead. Architects must carefully consider this balance, especially when designing systems expected to operate under high-load conditions. The results also highlight the importance of strategic deployment planning, as the performance of the application can vary significantly across different deployment configurations. Finally, the study underscores the need for tools and methods that can assist architects in making informed decisions about service granularity, as the optimal granularity may vary depending on specific application characteristics and performance objectives.

6 Related Work

The impact of microservice granularity decisions has been the focus of several studies in the literature, e.g., [21, 9, 8, 27, 2, 22, 18, 1, 25, 11]. According to Vera-Rivera et al. [27], most studies on this topic focus on the decomposition of monolithic systems into microservices, with less attention given to the empirical evaluation of granularity decisions in existing systems. In this regard, the studies most related to our work are those of Shadija *et al.* [21] and Costa & Ribeiro [2], as well as the recent investigations on modular monoliths by Su and Li [25] and Johnson et al. [11] discussed below.

The study by Shadija *et al.* [21] investigates the effect of service co-location on the latency of an academic admission application, simulating two deployment approaches: one with all services in a single container and another with services distributed across two containers. The results showed a negligible increase in latency for the two-container deployment compared to a single container. The study highlights that, although finer granularity of microservices may lead to an increase in the number of method invocations, this is manageable when microservices are within the same container. However, communication between containers introduces additional variables that can impact performance and reliability.

The work by Costa & Ribeiro [2] compares the performance of an e-commerce application developed following a traditional monolithic approach and two equivalent distributed versions based on different microservices communication patterns: direct communication and event-driven communication. The results showed that the monolithic approach had the lowest latency at all evaluated load levels, while the distributed approaches with direct communication and event-driven

communication performed worst, in that order. According to the authors, the latency associated with distributed systems, the eventual nature of responses to asynchronous operations, and the cost of data synchronization imposed a significant negative impact on the performance of the distributed solutions based on microservices compared to the monolithic solution.

Our work differs from those of Shadija *et al.* and Costa & Ribeiro in several aspects. First, while the work of Shadija *et al.* compares the performance of two deployment configurations of the target application, in one and two containers, our work compares the performance of eight deployment configurations of the target application, in one and two virtual machines. Second, while the work of Costa & Ribeiro compares the performance of a monolithic application with that of two equivalent distributed versions using different architectural patterns, our work compares the performance of a monolithic version of the target application with that of seven equivalent distributed versions, using the same communication topology among microservices. Finally, both works limited themselves to evaluating service granularity levels that were predetermined at the time of the applications' design, while our work benefits from the Service Weaver framework to evaluate multiple levels of granularity, defined at deployment time from the in-process coupling of different subsets of application components.

In terms of modular monoliths, Su and Li [25] conducted a systematic grey literature review, finding that modular monoliths combine the advantages of monoliths and microservices. They identified three frameworks for building modular monoliths: Service Weaver, Spring Modulith [24], and Light-hybrid-4j [12]. They also found that modular monoliths are increasingly adopted as an alternative to microservice decomposition, offering a balance between development velocity and operational complexity. Johnson et al. [11] examined Service Weaver through the lens of cloud-native principles, comparing its capabilities to traditional microservice architectures. They found that Service Weaver simplifies development and deployment processes but may lack certain features like polyglot support and fine-grained control over the system. Their work highlights the trade-offs involved in choosing between a modular monolith and microservices, emphasizing the importance of considering specific project requirements and constraints.

Our work builds upon the concept of a modular monolith by leveraging Service Weaver's unique capabilities to define service granularity at deployment time. By harnessing this flexibility, we can explore a wider range of deployment configurations and gain insights into the performance trade-offs associated with different levels of service granularity. This approach offers a more nuanced understanding of the relationship between service granularity and performance, contributing to the development of more effective strategies for designing and deploying microservice applications.

7 Conclusion and Future Work

This work investigated the impact of microservices deployment granularity on the performance of the distributed application Online Boutique, using the Service

Weaver operational framework. The application performance was analyzed under various deployment configurations and workloads, in environments with one and two virtual machines. Our findings demonstrated that both the granularity of deployment and the communication topology among microservices significantly influence application performance and scalability. These results underscore the importance of strategically balancing these factors to enhance application efficiency and maintainability.

Based on the insights gained from this study, several specific areas for future research can be identified, including the development of predictive models for granularity optimization, the exploration of more diverse application domains and granularity choices, and the assessment of the impact of different cloud-native platforms on microservice granularity design. Additionally, real-world application assessments, advanced deployment strategies, and resiliency aspects of service granularity should be further investigated to provide more concrete guidelines for architects and developers.

Data Availability To facilitate reproduction and potential extension of this work by the research community, all experimental artifacts and data generated are publicly available at `https://github.com/mendoncas/granularity-evaluation/`.

Acknowledgments. The generative AI tools ChatGPT-4 Turbo and GitHub Copilot were used to assist in structuring and revising this paper.

References

1. Andrade, B., Santos, S., Silva, A.R.: A Comparison of Static and Dynamic Analysis to Identify Microservices in Monolith Systems. In: European Conference on Software Architecture (ECSA). pp. 354–361. Springer (2023)
2. Costa, L., Ribeiro, A.N.: Performance Evaluation of Microservices Featuring Different Implementation Patterns. In: International Conference on Intelligent Systems Design and Applications (ISDA). pp. 165–176. Springer (2021)
3. Cui, J., Chen, P., Yu, G.: A Learning-based Dynamic Load Balancing Approach for Microservice Systems in Multi-cloud Environment. In: IEEE 26th International Conference on Parallel and Distributed Systems (ICPADS). pp. 334–341. IEEE (2020)
4. Dragoni, N., Giallorenzo, S., Lafuente, A.L., Mazzara, M., Montesi, F., Mustafin, R., Safina, L.: Microservices: Yesterday, Today, and Tomorrow. In: Present and Ulterior Software Engineering. Springer (2017)
5. Ghemawat, S., Grandl, R., Petrovic, S., Whittaker, M., Patel, P., Posva, I., Vahdat, A.: Towards Modern Development of Cloud Applications. In: Proceedings of the 19th Workshop on Hot Topics in Operating Systems. pp. 110–117 (2023)
6. Google: Service Weaver: A Programming Framework for Writing and Deploying Cloud Applications. `https://serviceweaver.dev` (2024), last access on June 25, 2024
7. Google Cloud: Online Boutique. `https://github.com/GoogleCloudPlatform/microservices-demo` (2023), last access on June 25, 2024

8. Hassan, S., Bahsoon, R., Kazman, R.: Microservice transition and its granularity problem: A systematic mapping study. Software: Practice and Experience **50**(9), 1651–1681 (2020)
9. Homay, A., Zoitl, A., de Sousa, M., Wollschlaeger, M., Chrysoulas, C.: Granularity Cost Analysis for Function Block as a Service. In: 2019 IEEE 17th International Conference on Industrial Informatics (INDIN). vol. 1, pp. 1199–1204. IEEE (2019)
10. Jamshidi, P., Pahl, C., Mendonça, N.C., Lewis, J., Tilkov, S.: Microservices: The Journey So Far and Challenges Ahead. IEEE Software **35**(3), 24–35 (2018)
11. Johnson, J., Kharel, S., Mannamplackal, A., Abdelfattah, A.S., Cerny, T.: Service Weaver: A Promising Direction for Cloud-native Systems? arXiv preprint arXiv:2404.09357 (2024)
12. Light: Light Hybrid 4j. `https://www.networknt.com/getting-started/light-hybrid-4j/` (2024), last access on June 25, 2024
13. Locust.io: Locust: An open source load testing tool. `https://locust.io/` (2024), last access on June 25, 2024
14. Molyneaux, I.: The Art of Application Performance Testing: From Strategy to Tools. O'Reilly Media (2014)
15. Newman, S.: Monolith to Microservices: Evolutionary Patterns to Transform your Monolith. O'Reilly Media (2019)
16. Oracle: VirtualBox. `https://www.virtualbox.org/` (2024), last access on June 25, 2024
17. Park, J., Choi, B., Lee, C., Han, D.: GRAF: A graph neural network based proactive resource allocation framework for SLO-oriented microservices. In: Proc. of the 17th International Conference on emerging Networking EXperiments and Technologies (CoNEXT). pp. 154–167 (2021)
18. Quéval, P.J., Zdun, U.: Extracting the Architecture of Microservices: An Approach for Explainability and Traceability. In: European Conference on Software Architecture (ECSA). pp. 346–353. Springer (2023)
19. Rota, G.C.: The Number of Partitions of a Set. The American Mathematical Monthly **71**(5), 498–504 (1964)
20. Saleh Sedghpour, M.R., Klein, C., Tordsson, J.: An Empirical Study of Service Mesh Traffic Management Policies for Microservices. In: ACM/SPEC Int. Conf. Performance Engineering (ICPE). pp. 17–27. AMC (2022)
21. Shadija, D., Rezai, M., Hill, R.: Microservices: Granularity vs. Performance. In: Companion Proceedings of the 10th International Conference on Utility and Cloud Computing (UCC). pp. 215–220 (2017)
22. Singh, S., Werle, D., Koziolek, A.: ARCHI4MOM: Using Tracing Information to Extract the Architecture of Microservice-Based Systems from Message-Oriented Middleware. In: European Conference on Software Architecture (ECSA). pp. 189–204. Springer (2022)
23. Soldani, J., Muntoni, G., Neri, D., Brogi, A.: The μTOSCA toolchain: Mining, analyzing, and refactoring microservice-based architectures. Software: Practice and Experience **51**(7), 1591–1621 (2021)
24. Spring: Spring Modulith. `https://spring.io/projects/spring-modulith` (2024), last access on June 25, 2024
25. Su, R., Li, X.: Modular Monolith: Is This the Trend in Software Architecture? arXiv preprint arXiv:2401.11867 (2024)
26. The Kubernetes Authors: Minikube. `https://minikube.sigs.k8s.io/docs/` (2024), last access on June 25, 2024
27. Vera-Rivera, F.H., Gaona, C., Astudillo, H.: Defining and measuring microservice granularity——a literature overview. PeerJ Computer Science **7**, e695 (2021)